

ASSIGNMENT-1

1. Introduction to Python and its Features.

- Easy to Learn and Use: Python's syntax is clear and closely resembles the English language, making it beginner-friendly.
- Interpreted Language: Python code is executed line by line, which simplifies debugging and allows for quick testing of code snippets.
- Dynamically Typed: Variables in Python do not require explicit type declarations. The type of a variable is determined at runtime.
- Object-Oriented: Python supports object-oriented programming (OOP), enabling developers to create reusable code through classes and objects.
- Extensive Standard Library: Python has a rich set of built-in libraries that support tasks such as file I/O, regular expressions, and networking, among others.
- Cross-platform: Python is platform-independent, meaning it can run on various operating systems, such as Windows, macOS, and Linux.
- Open Source: Python is free to use, distribute, and modify, which has led to a large and active community of developers contributing to its growth.
- Extensibility: Python can integrate with other programming languages, such as C, C++, and Java, to optimize performance-critical sections of code.

2. History and evolution of Python.

➤ **Origins (Late 1980s – Early 1990s)**

Python was created by Guido van Rossum in the late 1980s at Centrum Wiskunde & Informatica (CWI) in the Netherlands.

Development started in **December 1989** as a successor to the ABC language.

The first official release, **Python 0.9.0**, came out in **1991**.

It already included key features like exception handling, functions, and core data types.

➤ **Naming**

Despite what many think, Python was not named after the snake—it was inspired by the British comedy show Monty Python’s Flying Circus.

➤ **Python 1.x (1990s)**

Python 1.0 was released in **1994**.

Introduced functional programming tools like lambda, map, filter, and reduce.

Focus remained on clean, readable code.

➤ **Python 2.x Era (2000–2010)**

Python 2.0 was released in **2000**.

Major features included:

List comprehensions

Garbage collection

Unicode support (basic)

Backed by organizations like the Python Software Foundation (formed in 2001).

Python 2 became widely adopted but had design limitations that were hard to fix without breaking compatibility.

➤ **Python 3.x (2008–Present)**

Python 3.0 (also called “Python 3000”) was released in **2008**.

It was a **major redesign** that fixed inconsistencies:

Better Unicode handling

Clear separation of text vs. bytes

Improved syntax and standard library

Initially, adoption was slow due to incompatibility with Python 2, but over time it became the standard.

Official support for Python 2 ended in **2020**.

➤ **Modern Python (2010s–2020s)**

Python has grown rapidly and is now used in:

Web development (e.g., Django, Flask)

Data science and AI (NumPy, Pandas, TensorFlow)

Automation and scripting

Cybersecurity and DevOps

Its popularity surged due to:

Simple syntax

Strong community support

Extensive libraries and frameworks

➤ **Current Status**

Today, Python is one of the most widely used programming languages globally, heavily used by companies like Google, Microsoft, and Netflix. It continues to evolve with regular updates improving performance, typing, and developer experience.

3. Advantages of using Python over other programming languages.

- Easy to Learn and Use
 - Readable Syntax: Python's syntax is simple and similar to English, which makes it easy to read and understand. This is especially beneficial for beginners.
 - Minimalist Syntax: Python requires fewer lines of code compared to many other languages, making it easier to write and maintain.
- Cross-Platform Compatibility
 - Python is platform-independent, which means code written in Python can run on any operating system (Windows, Linux, macOS) without modification.
- Extensive Libraries and Frameworks
 - Python offers a wide range of pre-built libraries and frameworks for various tasks, including:
 - Web development: Django, Flask
 - Data Science & Machine Learning: Pandas, NumPy, TensorFlow, Scikit-learn
 - GUI development: Tkinter, PyQt
 - Automation: Selenium, OpenPyXL
- Large Community Support
 - Python has a vast and active community that provides support through forums, tutorials, and open-source contributions. This makes it easier to find solutions to problems and learn from others.
- Integration Capabilities
 - Python integrates easily with other programming languages like C, C++, and Java.
 - It also supports integration with technologies such as REST APIs and databases.
- Versatile Applications
 - Python can be used for a wide range of applications, including:
 - Web development
 - Data analysis and visualization
 - Machine learning and artificial intelligence
 - Scripting and automation
 - Game development
- Great for Prototyping and Development Speed
 - Python is known for its rapid prototyping capabilities, which allows developers to create and test applications faster

4. Installing Python and setting up the development environment.

- Download Python from Python
- Run installer → ✓ Check “Add to PATH” → Install
- Open terminal → type `python --version` (verify)
- Install editor like Visual Studio Code
- Install Python extension in VS Code
- Create file `hello.py` → write `print("Hello")`
- Run using `python hello.py`
- (Optional) Create virtual env: `python -m venv myenv` → activate

5. Writing and executing your first Python program.

1. Open a Code Editor

Use an editor like Visual Studio Code or any simple text editor.

2. Create a Python File

- Create a new file
- Save it as: hello.py

3. Write Your First Program

```
print("Hello, World!")
```

4. Save the File

5. Run the Program

Using Terminal / Command Prompt:

```
python hello.py
```

6. Output

Hello, World!

2. Programming Style

6. Understanding Python's PEP 8 guidelines:

PEP 8 – Python Style Guide Python's official style guide, PEP 8 (Python Enhancement Proposal 8), defines conventions for writing clean and consistent Python code. It covers several aspects such as naming conventions, indentation, line length, and more.

- Indentation: Use 4 spaces per indentation level (avoid using tabs).
- Maximum Line Length: Limit all lines to a maximum of 79 characters (72 for docstrings).
- Naming Conventions Python has specific conventions for naming variables, functions, classes, and modules. These conventions help make code more readable and consistent.
- Classes: Use CamelCase (first letter capitalized, no underscores)

7. Indentation, comments, and naming conventions in Python.

1. Indentation in Python

- Python uses indentation (spaces/tabs) instead of {} to define blocks.
- Usually 4 spaces are used.
- All lines in a block must have the same indentation.

Example:

```
if True:
    print("Hello") # indented
print("Outside")  # not indented
```

2. Comments in Python

- Comments are used to explain code.
- They are ignored by Python.

Types:

- Single-line comment → starts with #

```
# This is a comment
print("Hi")
```

- Multi-line comment → use triple quotes

```
"""
This is
multi-line comment
"""
```

3. Naming Conventions in Python

- Follow rules for clean and readable code.

Common rules:

- Use lowercase with underscores for variables and functions
 - my_variable, calculate_sum()
- Use CamelCase for class names
 - MyClass
- Use UPPERCASE for constants
 - PI = 3.14
- Names should be meaningful
 - x → total_marks

8. Writing readable and maintainable code.

Writing readable and maintainable code is very important in programming. It helps others understand the code easily and makes it simple to update or fix in the future.

First, using meaningful names for variables and functions makes the code clear. For example, names like `total_marks` are better than `x` because they explain the purpose.

Second, following a consistent coding style improves readability. Proper indentation, spacing, and naming conventions make the code look clean and organized.

Third, code should be simple and divided into small functions. Each function should perform one task. This makes the program easier to understand and reuse.

Comments also play an important role. They help explain why the code is written, but too many unnecessary comments should be avoided.

Another important point is avoiding repetition. Instead of writing the same code again, functions or loops should be used.

Finally, proper error handling and testing make the code reliable and easy to maintain.

In conclusion, readable and maintainable code saves time, reduces errors, and improves collaboration among programmers.

3. Core Python Concepts

9.Understanding data types: integers, floats, strings, lists, tuples, dictionaries, sets:

1. int (Integer): Represents whole numbers, positive or negative, without decimals.
2. float (Floating-point): Represents real numbers or numbers with a fractional part.
3. str (String): Represents a sequence of characters enclosed in single, double, or triple quotes
4. list: Represents an ordered, mutable collection of items. Lists can hold different types of data (integers, strings, other lists, etc.)..
5. tuple: Represents an ordered, immutable collection of items. Once created, its elements cannot be changed.
6. dict (Dictionary): Represents a collection of key-value pairs. Each key is unique, and the associated value can be any data type.
7. set: Represents an unordered collection of unique elements. Sets do not allow duplicate values.

10. Python variables and memory allocation.

- a. Variable is a name which is contain some value.
- b. Variable Rules: Variable Names Must Begin with a Letter or Underscore:
- c. The first character of a variable name must be a letter (a-z, A-Z) or an underscore (_).
- d. For example, age, name, _value, and score1 are valid variable names.
- e. Variable Names Can Contain Letters, Numbers, and Underscores: After the first character, the variable name can include letters (a-z, A-Z), digits (0-9), and underscores (_). For example, age_1, student_score, and value_2 are valid variable names.
- f. Variable Names Are Case-Sensitive: ○ Python treats uppercase and lowercase letters as different. For example, age and Age are two different variables.
- g. age, Age, and AGE are considered distinct. Variable Names Cannot Be Reserved Keywords:
- h. Python has a set of reserved keywords (like class, for, if, True, False, def, etc.) that cannot be used as variable names.
- i. For example, if, for, and while are not allowed as variable names. Variable Names Should Be Meaningful:
- j. While not a strict rule, it's good practice to choose meaningful names that convey the purpose of the variable. For example, age for a person's age,

memory allocation:

Step 1: Variable Creation

- When you write a variable, Python starts the process

```
x = 10
```

Step 2: Object Creation

- Python creates an object (value 10) in memory

Step 3: Memory Allocation

- Memory is automatically allocated to store the object

Step 4: Reference Assignment

- Variable x is linked (reference) to that object
□ $x \rightarrow 10$

Step 5: Reuse of Memory (if possible)

- If same value exists, Python may reuse memory

a = 10

b = 10

- Both may point to same memory location

Step 6: Reassignment

- If value changes, new memory is allocated

x = 20

- x now points to a new object

Step 7: Garbage Collection

- If no variable refers to old object, Python removes it automatically

11. Python operators: arithmetic, comparison, logical, bitwise.

1. Arithmetic Operators (Math operations)

Used for basic calculations.

Operator	Meaning	Example
+	Addition	$5 + 2 = 7$
-	Subtraction	$5 - 2 = 3$
*	Multiplication	$5 * 2 = 10$
/	Division	$5 / 2 = 2.5$
%	Modulus (remainder)	$5 \% 2 = 1$
**	Power	$2 ** 3 = 8$

2. Comparison Operators (Check conditions)

Return True or False

Operator	Meaning	Example
==	Equal to	$5 == 5 \rightarrow \text{True}$
!=	Not equal	$5 != 3 \rightarrow \text{True}$
>	Greater than	$5 > 2 \rightarrow \text{True}$
<	Less than	$5 < 2 \rightarrow \text{False}$
>=	Greater or equal	$5 >= 5 \rightarrow \text{True}$
<=	Less or equal	$3 <= 5 \rightarrow \text{True}$

3. Logical Operators (Combine conditions)

Operator	Meaning	Example
and	Both must be True	$\text{True and False} \rightarrow \text{False}$

Operator	Meaning	Example
or	One must be True	True or False $\rightarrow$ True
not	Reverse result	not True $\rightarrow$ False

\

4. Bitwise Operators (Work on binary numbers)

Used for low-level operations

Operator	Meaning	Example
&	AND	5 & 3 = 1
	OR	
^	XOR	5 ^ 3 = 6
~	NOT	~5 = -6

4. Conditional Statements

12 .Introduction to conditional statements: if, else, elif.

- a) Conditional statements in Python are used to **control the flow of a program** based on certain conditions. They help the program make decisions and execute different blocks of code accordingly.
- b) The if statement is used to test a condition. If the condition is **True**, the block of code inside the if statement is executed.
- c) The else statement is used when the condition in the if statement is **False**. It provides an alternative block of code to execute.
- d) The elif (else if) statement is used to check **multiple conditions**. If the first condition is false, Python checks the next elif condition, and this continues until a true condition is found.
- e) These statements rely on **Boolean expressions** (True or False) and use proper indentation to define blocks of code.
- f) In conclusion, conditional statements make programs **intelligent and flexible** by allowing different actions based on different situations.

13. Nested if-else conditions

- a) Nested if-else is a type of conditional statement where an if-else statement is placed inside another if or else block. It is used when a decision depends on multiple conditions.
- b) In nested if-else, the program first checks the outer condition. If it is true, then the inner condition is evaluated. Based on the result of the inner condition, the appropriate block of code is executed. If the outer condition is false, the else part of the outer block is executed.
- c) This structure helps in handling complex decision-making situations where one condition depends on another.
- d) Proper indentation is very important in nested if-else statements because it defines the structure and flow of the program.
- e) However, excessive use of nested if-else can make the code complex and difficult to read. Therefore, it should be used carefully.

5.Looping (For, While)

14. Introduction to for and while loops.

1. for Loop

- Used when the number of iterations is known
- Iterates over a sequence (like list, string, range)

Syntax:

```
for variable in sequence:  
    # code block
```

Example:

```
for i in range(1, 6):  
    print(i)
```

2. while Loop

- Used when the number of iterations is not known
- Runs as long as the condition is True

Syntax:

```
while condition:  
    # code block
```

Example:

```
i = 1  
while i <= 5:  
    print(i)  
    i += 1
```

15. How loops work in Python.

1. Initialization

- A variable is set to a starting value
□ `i = 1`

2. Condition Check

- Python checks whether the condition is True or False

3. Execute Code Block

- If condition is True, the loop body runs

4. Update (Increment/Decrement)

- The loop variable is updated
□ `i = i + 1`

5. Repeat

- Steps 2–4 repeat until condition becomes False

6. Exit Loop

- When condition is False, loop stops

16. Using loops with collections (lists, tuples, etc.)

1. Using for loop with List

```
numbers = [1, 2, 3, 4]
```

```
for num in numbers:  
    print(num)
```

- ☐ Prints each element in the list

2. Using for loop with Tuple

```
colors = ("red", "green", "blue")
```

```
for color in colors:  
    print(color)
```

- ☐ Works same as list

3. Using for loop with String

```
text = "Python"
```

```
for ch in text:  
    print(ch)
```

4. Using for loop with Dictionary

```
student = {"name": "Mira", "age": 21}
```

```
for key in student:  
    print(key, ":", student[key])
```

- ☐ Access key and value

5. Using while loop with List

```
numbers = [10, 20, 30]
```

```
i = 0
```

```
while i < len(numbers):  
    print(numbers[i])  
    i += 1
```

6. Generators and Iterators

17. Understanding how generators work in Python.

What is a Generator?

A generator is a special function that produces values one at a time instead of returning all values at once.

Key Idea

- Uses yield instead of return
- Saves memory (does not store all values)
- Generates values on demand

How Generators Work (Step-wise)

1. Function is defined with yield
2. When called → it does not run immediately
3. It returns a generator object
4. Each time you use next() or loop → it runs until yield
5. Pauses and remembers its state
6. Continues from where it stopped.

18. Difference between yield and return.

Feature	return	yield
Purpose	Ends function and returns value	Pauses function and gives value
Execution	Function stops completely	Function resumes later
Memory	Stores all results at once	Generates values one by one
Use	Normal functions	Generator functions
Reusability	Cannot continue after return	Continues from last point

19. Understanding iterators and creating custom iterators.

What is an Iterator?

An iterator is an object that allows you to traverse (go through) elements one by one.

- Uses two methods:
 - `__iter__()` → returns the iterator object
 - `__next__()` → returns next value

How Iterator Works

1. Convert a collection into iterator using `iter()`
2. Use `next()` to get elements one by one
3. When no elements left → raises `StopIteration`

Example (Built-in Iterator)

```
numbers = [10, 20, 30]
```

```
it = iter(numbers)
```

```
print(next(it)) # 10
```

```
print(next(it)) # 20
```

```
print(next(it)) # 30
```

7. Functions and Methods

20. Defining and calling functions in Python.

1. Definition of Function

- A function is a block of organized, reusable code used to perform a single task.
- It helps make programs modular and easy to manage.

2. Function Declaration (def)

- Functions are defined using the def keyword.
- Syntax:

```
def function_name():  
    pass
```

3. Function Name Rules

- Must follow variable naming rules:
 - Can contain letters, digits, underscore (_)
 - Cannot start with a number
 - Should be meaningful (e.g., calculate_sum)

4. Parameters and Arguments

- Parameters: Variables listed in function definition.
- Arguments: Values passed when calling the function.
- Types:
 - Positional arguments
 - Keyword arguments
 - Default arguments

5. Function Call

- A function is executed by calling its name followed by parentheses ().
- Example:

```
function_name()
```

6. Return Statement

- return is used to send a value back from the function.
- If no return is given, function returns None by default.

7. Types of Functions

- Built-in functions (e.g., print(), len())
- User-defined functions
- Anonymous functions (lambda)

8. Scope of Variables

- Local variables: Defined inside a function
- Global variables: Defined outside all functions
- Local variables cannot be accessed outside the function.

9. Advantages of Functions

- Code reusability
- Improves readability
- Reduces code duplication
- Easier debugging and maintenance

10. Function Execution Flow

- Function is defined → Function is called → Code inside function executes → Result is returned (if any)

21. Function arguments (positional, keyword, default)

1. Positional Arguments

- Arguments are passed in the same order as parameters in the function definition.
- Order matters.

Example:

```
def display(name, age):  
    print(name, age)
```

```
display("Alice", 25)
```

□ Here, "Alice" → name, 25 → age

2. Keyword Arguments

- Arguments are passed using parameter names.
- Order does not matter.

Example:

```
def display(name, age):  
    print(name, age)
```

```
display(age=25, name="Alice")
```

□ Values are assigned based on keywords, not position.

3. Default Arguments

- Parameters are given default values in the function definition.
- If no value is passed, the default is used.

Example:

```
def greet(name="Guest"):  
    print("Hello", name)
```

```
greet()      # Uses default value  
greet("Alice")
```

22. Scope of variables in Python.

1. Types of Scope

☐ Local Scope

- Variables declared inside a function.
- Accessible only within that function.

Example:

```
def func():  
    x = 10 # local variable  
    print(x)  
  
func()  
# print(x) ✗Error (outside scope)
```

☐ Global Scope

- Variables declared outside all functions.
- Accessible throughout the program.

Example:

```
x = 20 # global variable  
  
def func():  
    print(x)  
  
func()  
print(x)
```

☐ Enclosing Scope (Non-local Scope)

- Found in nested functions.
- Variable belongs to the outer function.

Example:

```
def outer():  
    x = 15  
    def inner():  
        print(x)  
    inner()  
  
outer()
```

□ Built-in Scope

- Variables/functions that are predefined in Python.
- Always available.

Examples:

- `print()`
- `len()`
- `range()`

2. LEGB Rule

Python follows the LEGB rule to search for variables:

1. $L \rightarrow$ Local
2. $E \rightarrow$ Enclosing
3. $G \rightarrow$ Global
4. $B \rightarrow$ Built-in

□ Python checks in this order when accessing a variable.

3. Global Keyword

- Used to modify a global variable inside a function.

Example:

```
x = 10
```

```
def func():  
    global x  
    x = 50
```

```
func()  
print(x) # 50
```

4. Nonlocal Keyword

- Used in nested functions to modify enclosing variables.

Example:

```
def outer():  
    x = 10  
    def inner():  
        nonlocal x  
        x = 20  
    inner()  
    print(x) outer()
```

23. Built-in methods for strings, lists, etc.

1. String Methods

Strings are immutable (cannot be changed), but methods return new strings.

□ Common String Methods

- `lower()` → converts to lowercase
- `upper()` → converts to uppercase
- `title()` → capitalizes each word
- `strip()` → removes whitespace
- `replace(old, new)` → replaces substring
- `split()` → splits string into list
- `find()` → returns index of substring
- `count()` → counts occurrences

Example:

```
text = " Hello World "  
print(text.strip().upper())
```

2. List Methods

Lists are mutable (can be changed).

□ Common List Methods

- `append(x)` → adds element at end
- `insert(i, x)` → inserts at position
- `remove(x)` → removes element
- `pop()` → removes last element
- `sort()` → sorts list
- `reverse()` → reverses list
- `count(x)` → counts occurrences
- `copy()` → creates a copy

Example:

```
nums = [3, 1, 2]  
nums.sort()  
print(nums)
```

3. Tuple Methods

Tuples are immutable.

□ Common Tuple Methods

- `count(x)` → counts occurrences
- `index(x)` → finds index

4. Set Methods

Sets store unique values.

□ Common Set Methods

- `add(x)` → adds element
- `remove(x)` → removes element
- `union()` → combines sets
- `intersection()` → common element s
- `difference()` → subtracts sets

5. Dictionary Methods

Dictionaries store key-value pairs.

□ Common Dictionary Methods

- `keys()` → returns all keys
- `values()` → returns all values
- `items()` → returns key-value pairs
- `get(key)` → gets value safely
- `update()` → updates dictionary
- `pop(key)` → removes key

8. Control Statements (Break, Continue, Pass)

24. Understanding the role of break, continue, and pass in Python loops.

1. break Statement

- Terminates the entire loop immediately.
- Control transfers to the first statement after the loop.
- Usually used when a specific condition is met.

□ Key Idea: *Exit the loop completely*

2. continue Statement

- Skips the current iteration of the loop.
- Moves control to the next iteration.
- Does not stop the loop completely.

□ Key Idea: *Skip current step, continue loop*

3. pass Statement

- A null statement (does nothing).
- Used as a placeholder where a statement is syntactically required.
- Common in empty loops, functions, or conditional blocks.

9. String Manipulation

25. Understanding how to access and manipulate strings.

1. Accessing Strings

☐ Indexing

- Each character has a position (index).
- Index starts from 0.

☐ Example concept:

- First character → index 0
- Last character → index -1 (negative indexing)

☐ Slicing

- Used to access a range of characters.
- Syntax: string[start : end : step]

☐ Key Points:

- start → beginning index
- end → ending index (excluded)
- step → interval between characters

2. String Manipulation

☐ Concatenation

- Joining two or more strings using +.

☐ Repetition

- Repeating a string using *.

☐ Membership

- Check if a substring exists using in / not in.

3. String Methods

- Strings provide built-in methods for manipulation:
 - Case conversion: `upper()`, `lower()`
 - Removing spaces: `strip()`
 - Replacing text: `replace()`
 - Splitting: `split()`
 - Searching: `find()`, `count()`

□ These methods return new strings (original string remains unchanged).

4. Immutability of Strings

- Strings cannot be modified directly.
- Any operation creates a new string.

5. Iterating Through Strings

- Strings can be accessed using loops (for loop).
- Each character can be processed one by one.

6. Escape Sequences

- Used to represent special characters:
 - `\n` → new line
 - `\t` → tab
 - `\\` → backslash

7. String Formatting

- Used to insert values into strings:
 - `format()` method
 - f-strings (`f" "`)

26. Basic operations: concatenation, repetition, string methods (upper(), lower(), etc.).

1. Concatenation

- Concatenation means joining two or more strings.
- Performed using the + operator.

□ Key Points:

- Combines strings into one.
- Both operands must be strings.
- Creates a new string.

2. Repetition

- Repetition means repeating a string multiple times.
- Performed using the * operator.

□ Key Points:

- Multiplies string by an integer.
- Returns a new repeated string.

3. String Methods

- Strings provide built-in methods to manipulate data.
- Since strings are immutable, methods return new strings.

□ Case Conversion Methods

- upper() → converts all characters to uppercase
- lower() → converts all characters to lowercase
- title() → capitalizes first letter of each word

□ Removing Spaces

- strip() → removes spaces from both sides
- lstrip() → removes from left
- rstrip() → removes from right

□ Searching Methods

- find() → returns index of substring
- count() → counts occurrences

27. String slicing.

- String slicing is used to extract a part (substring) of a string.
- It allows access to a range of characters instead of a single character.

□ Syntax

`string[start : end : step]`

□ Explanation of Parameters

- start → index where slicing begins
- end → index where slicing stops (excluded)
- step → interval between characters

□ Key Concepts

□ Default Values

- If start is omitted → starts from beginning
- If end is omitted → goes till end of string
- Default step is 1

□ Indexing Rules

- Index starts from 0
- Negative indexing is allowed:
 - -1 → last character
 - -2 → second last

□ Step Value

- Positive step → moves forward
- Negative step → moves backward (reverse direction)

□ Types of Slicing

1. Basic Slicing

- Extract characters between two positions.

2. Partial Slicing

- Only start or end is given.

3. Full Slicing

- `[:]` returns the entire string.

10. Advanced Python (map(), reduce(), filter(), Closures and Decorators)

28. How functional programming works in Python.

1. First-Class Functions

- Functions in Python are treated as objects.
- They can be:
 - Assigned to variables
 - Passed as arguments
 - Returned from other functions

□ Key Idea: Functions behave like data.

2. Pure Functions

- A pure function:
 - Always returns the same output for the same input
 - Has no side effects (does not modify external data)

□ Makes code predictable and easy to test.

3. Immutability

- Functional programming prefers immutable data.
- Data is not changed; instead, new data is created.

□ Example: Strings and tuples are immutable.

4. Higher-Order Functions

- Functions that:
 - Take other functions as arguments
 - Return functions as results

□ Examples:

- map() → applies a function to all elements
- filter() → filters elements based on condition
- reduce() → reduces data to a single value

5. Lambda Functions

- Small, anonymous (unnamed) functions.
- Defined using lambda keyword.

□ Used for short, simple operations.

6. Recursion

- A function calling itself.
- Used instead of loops in functional style.

7. Avoiding State and Side Effects

- Functional programming avoids:
 - Changing global variables
 - Modifying data in-place

☐ Focus is on data transformation.

8. Benefits of Functional Programming

- Code is clean and concise
- Easier to debug and test
- Supports parallel processing
- Reduces errors due to immutability

29. Using map(), reduce(), and filter() functions for processing data.

1. map() Function

- Applies a function to each element of an iterable.
- Returns a map object (can be converted to list).

□ Syntax:

```
map(function, iterable)
```

□ Key Idea:

- Transforms data (one-to-one mapping).

□ Example Concept:

- Convert all elements (e.g., square each number).

2. filter() Function

- Selects elements from an iterable based on a condition.
- Returns a filter object.

□ Syntax:

```
filter(function, iterable)
```

□ Key Idea:

- Keeps only elements where condition is True.

□ Example Concept:

- Extract even numbers from a list.

3. reduce() Function

- Reduces all elements of an iterable to a single value.
- Available in the functools module.

□ Syntax:

```
from functools import reduce  
reduce(function, iterable)
```

30. Introduction to closures and decorators.

Definition

- A closure is a function that remembers variables from its enclosing scope even after the outer function has finished execution.

□ Key Concept

- A closure is created when:
 - There is a nested function
 - The inner function uses variables from the outer function
 - The outer function returns the inner function

□ Characteristics

- Preserves the state of variables
- Uses enclosing scope (non-local variables)
- Supports data hiding

□ Why Use Closures?

- To maintain state without using global variables
- To create function factories
- Useful in functional programming

□ Decorators in Python

□ Definition

- A decorator is a function that modifies or extends the behavior of another function without changing its code.

□ Key Concept

- Decorators use closures.
- They wrap a function inside another function.

□ Syntax

```
@decorator_name  
def function():  
    pass
```

□ How It Works

- A decorator takes a function as input
- Adds some functionality
- Returns a new modified function

□ Uses of Decorators

- Logging
- Authentication
- Timing functions
- Access control